

VERIQO Replay Specification

Replay re-derives a case's Decision (and Action, when present) from its own recorded inputs and reports whether the results converge byte-identically with what was originally recorded.

Implementations: `Store.Replay` (`pkg/commercial/api/store.go`) and `verticalslice.Replay`.

1. What replay is for

Replay answers: **"if we ran this decision again, would we get the same answer — bit for bit?"**

A `converged: true` result establishes that the recorded Decision hash is genuinely a function of the recorded inputs, not an arbitrary value someone could have written in. A `converged: false` result is a substantive finding: either the inputs changed, the evidence changed, or the engine's behaviour changed between the two runs. None of those is acceptable silently.

This is what makes an old case defensible. A challenger years later can be shown not merely a stored conclusion, but that the conclusion still follows from the inputs.

2. What replay actually re-runs

`GET /v1/cases/{id}/replay` re-executes, from the case's stored inputs:

1. Rebuild the hypothesis set (hypothesis, supporting and contradicting evidence citations).
2. Rebuild the Finding from the recorded `FindingInput`.
3. Re-authorize the Finding **against the live evidence manifests** — the same grounding gate the original decision passed.
4. Re-make the Decision from the recorded outcome, rationale, and tick.
5. If an action exists, re-authorize it from the recorded action inputs against the *replayed* Decision.

Then compare hashes:

```
{
  "original_decision_hash": "...",
  "replayed_decision_hash": "...",
  "original_action_authorization_hash": "...",
  "replayed_action_authorization_hash": "...",
  "converged": true
}
```

`converged` is true only when **every** compared pair matches exactly.

3. Why it is deterministic

Determinism is a design property, not a lucky outcome:

- **No wall-clock reads.** Every time value is a caller-supplied logical tick, stored with the case. The engine never calls the system clock.
- **No randomness** anywhere in the decision path.

- **Canonical serialization (JCS)** before hashing, so field ordering and encoding cannot vary between runs or machines.
- **Hash-linked, append-only ledger** — the same call sequence produces the same chain.

The same properties are what make the durable store's WAL replay sound: reconstructing state by replaying recorded inputs through the real logic yields byte-identical hashes, not merely equivalent-looking state.

4. Scope — stated precisely

Replay's honest boundary matters, so it is stated rather than implied.

Replay DOES re-derive: the Finding, its authorization/grounding against evidence, the Decision, and the Action authorization.

Replay does NOT re-derive the evidence manifests themselves. The manifests are treated as durable infrastructure that already exists — replay runs *against* them, exactly as the original decision did. This is deliberate: the manifests are the ground truth being cited, not an intermediate result. Their integrity is checked by a different mechanism (manifest hash verification and custody-chain verification, both of which the verifier and `POST /v1/evidence/{id}/verify` perform independently).

So: replay proves *the decision follows from the evidence*. Manifest verification proves *the evidence is unaltered*. Both together are the complete claim; neither alone is.

5. Two distinct replay contexts

These are frequently conflated, so they are named separately:

(a) Live replay against a running Store — `GET /v1/cases/{id}/replay`. Re-derives the decision from stored inputs against live manifests, as described above. Requires the case to be decided (422 otherwise) and the tenant to match.

(b) Package-level verification — the standalone verifier on an exported Machine Package. This re-verifies the ledger's own hash chain from genesis and cross-references the dossier's claimed decision and authorization hashes against what the independently-parsed ledger actually recorded (lineage).

The package verifier does not perform full input-level replay, and does not claim to: a Machine Package deliberately excludes the original decision and action *inputs*. Full re-derivation is `Store.Replay`'s job, against the live Store. The verifier's honest claim is lineage and chain integrity, not re-computation from scratch.

6. Interpreting a divergence

`converged: false` should page a human. Diagnose in this order:

1. **Was the evidence altered or superseded?** Run `POST /v1/evidence/{id}/verify` on each cited item. A failed manifest or custody check explains the divergence and is the more serious finding.
2. **Did the software version change?** A behavioural change in the decision path between the original run and the replay would diverge every case, not one. Check whether other cases also diverge.
3. **Was the store restored from a backup or replayed WAL?** Restoration is designed to reproduce byte-identical state; a divergence here indicates a real recovery defect and should be reported.

A single case diverging while others converge points at that case's evidence. All cases diverging points at the deployment.

The `replay_failures` counter in `GET /v1/metrics` increments on divergence, so this is alertable without polling every case.

7. Guarantees and limits

Guaranteed: given unchanged inputs and unchanged finalized evidence, replay converges byte-identically, on any machine, at any later time, including after a crash-recovery restart or a restore from backup.

Not guaranteed: that a replay performed under a *different software version* converges. Determinism is within a version of the decision engine. Cross-version replay is a stronger property that would require versioned engine semantics; it is not claimed today. For long-horizon defensibility, retain the exported dossier package — it is self-contained and independently checkable regardless of what version is running later.